Software Requirements Specification

For

Banking System

Version 1.0 approved

Prepared by:- Vatsal Mehta
Ruchit Vaghela
S.P.C.E.T.

04-05-2026

Table of Contents

PRACTICAL 3

Aim: To perform the user's view analysis: Use case diagram

- **Use Case Diagram:**
  - A use case diagram is used to represent the dynamic behaviour of a system. It encapsulates the system's functionality by incorporating use cases, actors, and their relationships. It models the tasks, services, and functions required by a system/subsystem of an application. It depicts the high-level functionality of a system and also tells how the user handles a system.
- **The following topics describe model elements in use-case diagrams:**
  - **Use cases:**

    A use case describes a function that a system performs to achieve the user's goal. A use case must yield an observable result that is of value to the user of the system. Each use case describes a particular goal for the user and how the user interacts with the system to achieve that goal. The use case describes all possible ways that the system can achieve, or fail to achieve, the goal of the user.

    You can use use cases for the following purposes:

    - Determine the requirements of the system
    - Describe what the system should do
    - Provide a basis for testing to ensure that the system works as intended

    As the following figure illustrates, a use case is displayed as an oval that contains the name of the use case.

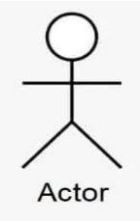
    

    You can add the following features to use cases:

    - Attributes that identify the properties of the objects in a use case
    - Operations that describe the behaviour of objects in a use case and how they affect the system
    - Documentation that details the purpose and flow of events in a use case
  - **Actors**

- An actor represents a role of a user that interacts with the system that you are modeling. The user can be a human user, an organization, a machine, or another external system. You can represent multiple users with a single actor and a single user can have the role of multiple actors.

  Actors are external to the system. They can initiate the behaviour described in the use case or be acted upon by the use case. Actors can also exchange data with the system. Each actor has a unique name that describes the role of the user who interacts with the system. As the following figure illustrates, an actor is displayed as a line drawing of a person.



➢ **Relationships in use-case diagrams**

- In UML, relationships are connections between model elements.
- Use cases are also connected to each other in different kinds of relationships.
- The relationship between two use cases basically models the dependencies between two use cases.
- By reusing existing use cases using different types of relationships, the overall effort required to develop the system is reduced.
- UML relationships are grouped into the following categories:

1. Association Relationships

An association is a relationship between two classifiers, such as an actor and use cases, that describes the cause of the relationship and the rules that govern it. An association is a relationship between an actor and a business use case. It indicates that an actor can use the functionality of the business system.

Fig: Association Relationships

2. Generalization Relationships

A generalization relation is a relationship in which one model element (child) is based on another model element (parent). Generalization relations are used in class diagrams, component diagrams, deployment diagrams, and use case diagrams to indicate that the child element accepts all the attributes, operations, and relationships defined in the parent element.

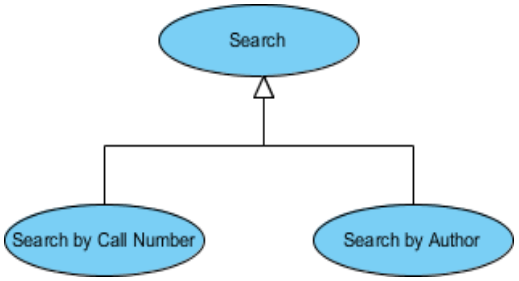


Fig: Generalization Relationship

3. Include Relationships

In UML modeling, a Include relationship is a relationship in which one use case (base use case) contains the functionality of another use case (inclusion use case). A containment relationship supports the reuse of functionality in the use case model.

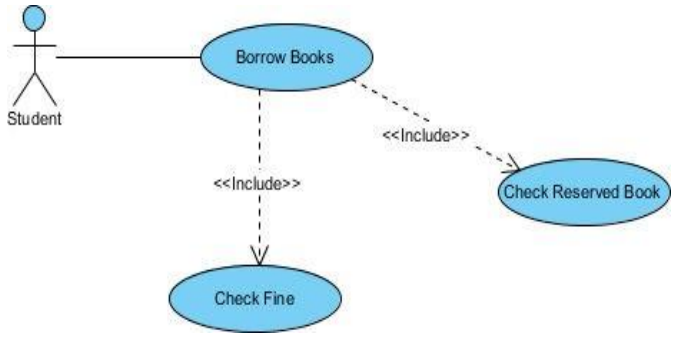
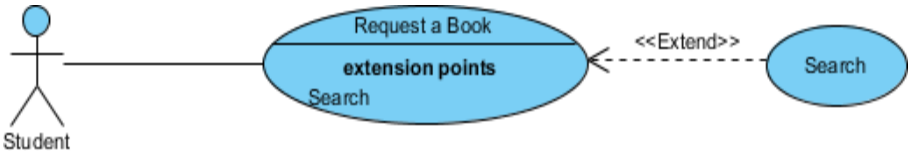


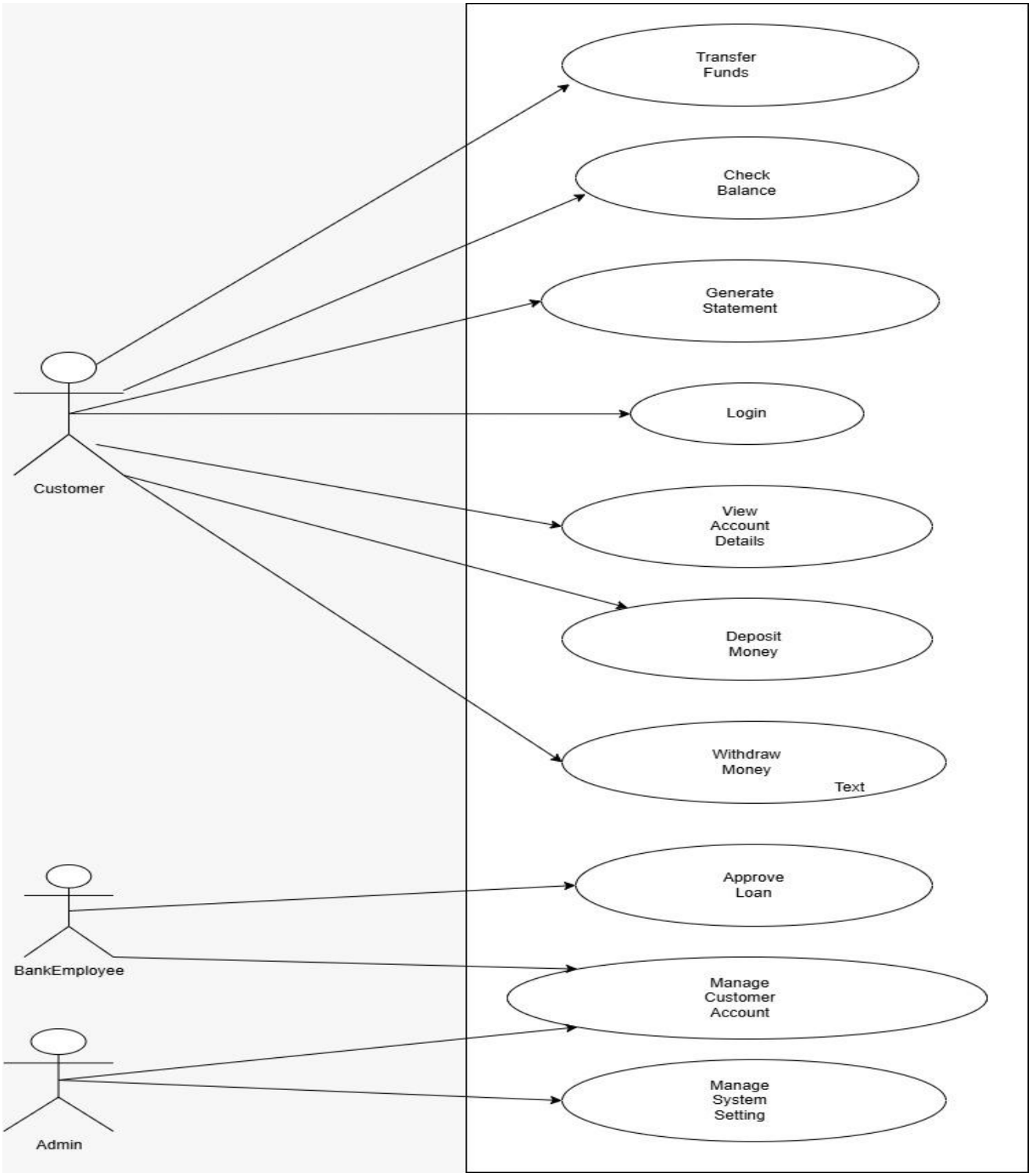
Fig: Include Relationships

4. Extending relationships

In UML modeling, you can use an extend relationship to specify that one use case (extension) extends the behaviour of another use case (base). This type of relationship reveals details about the system or application that are usually hidden in the use case.



Use-case diagram for Banking System



PRACTICAL 4

Aim: To draw the structural view diagram: Class diagram, Object diagram

Class Diagram:

The class diagram depicts a static view of an application. It represents the types of objects residing in the system and the relationships between them. A class consists of its objects, and also it may inherit from other classes. A class diagram is used to visualize, describe, document various different aspects of the system, and also construct executable software code.

It shows the attributes, classes, functions, and relationships to give an overview of the software system. It constitutes class names, attributes, and functions in a separate compartment that helps in software development. Since it is a collection of classes, interfaces, associations, collaborations, and constraints, it is termed as a structural diagram.

The main purpose of class diagrams is to build a static view of an application. It is the only diagram that is widely used for construction, and it can be mapped with object-oriented languages. It is one of the most popular UML diagrams. Following are the purpose of class diagrams given below:

- It analyses and designs a static view of an application.
- It describes the major responsibilities of a system.
- It is a base for component and deployment diagrams.
- It incorporates forward and reverse engineering.

Components of a Class Diagram:

A class notation consists of three parts:

Upper Section: The upper section encompasses the name of the class. A class is a representation of similar objects that shares the same relationships, attributes, operations, and semantics. Some of the following rules that should be taken into account while representing a class are given below:

- Capitalize the initial letter of the class name.

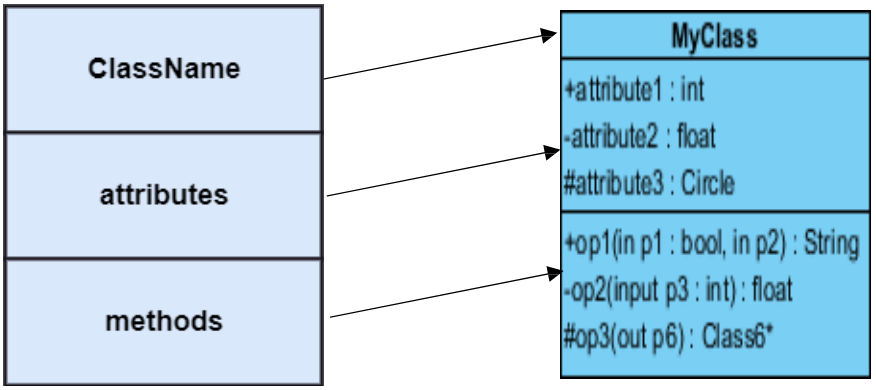
Place the class name in the center of the upper section.

- A class name must be written in bold format.
- The name of the abstract class should be written in italics format.

Middle Section: The middle section constitutes the attributes, which describe the quality of the class. The attributes have the following characteristics:

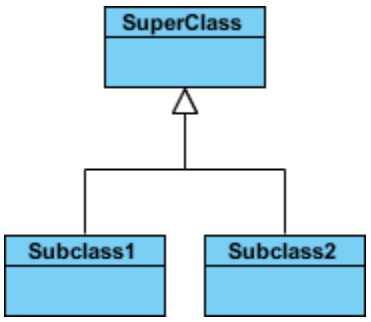
- The attributes are written along with its visibility factors, which are public (+), private (-), protected (#), and package (~).
- The accessibility of an attribute class is illustrated by the visibility factors.
- A meaningful name should be assigned to the attribute, which will explain its usage inside the class.

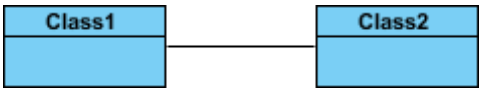
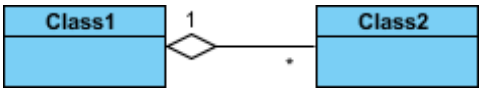
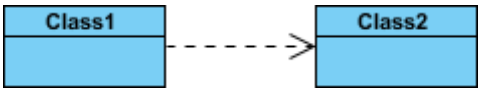
Lower Section: The lower section contains methods or operations. The methods are represented in the form of a list, where each method is written in a single line. It demonstrates how a class interacts with data.



Class Relationships:

A class may be involved in one or more relationships with other classes. A relationship can be one of the following types

Relationship Type	Graphical Representation
Inheritance (or Generalization): - Represents an "is-a" relationship. - An abstract class name is shown in italics. - SubClass1 and SubClass2 are specializations of Super Class. - A solid line with a hollow arrowhead that point from the child to the parent class	

Simple Association: - A structural link between two peer classes. - There is an association between Class1 and Class2 - A solid line connecting two classes	
Aggregation: - A special type of association. It represents a "part of" relationship. - Class2 is part of Class1. - Many instances (denoted by the *) of Class2 can be associated with Class1. - Objects of Class1 and Class2 have separate lifetimes. - A solid line with an unfilled diamond at the association end connected to the class of composite	
Composition: - A special type of aggregation where parts are destroyed when the whole is destroyed. - Objects of Class2 live and die with Class1. - Class2 cannot stand by itself. - A solid line with a filled diamond at the association connected to the class of composite	
Dependency: - Exists between two classes if the changes to the definition of one may cause changes to the other (but not the other way around). - Class1 depends on Class2 - A dashed line with an open arrow	

Usage of Class diagrams

The class diagram is used to represent a static view of the system. It plays an essential role in the establishment of the component and deployment diagrams. Class diagrams can be used for the following purposes:

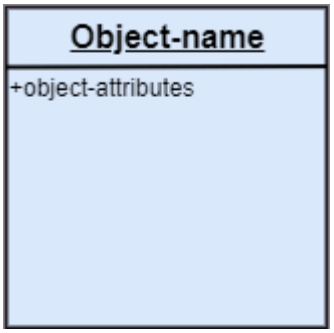
1. To describe the static view of a system.
2. To show the collaboration among every instance in the static view.
3. To describe the functionalities performed by the system.
4. To construct the software application using object-oriented languages.

Object Diagram

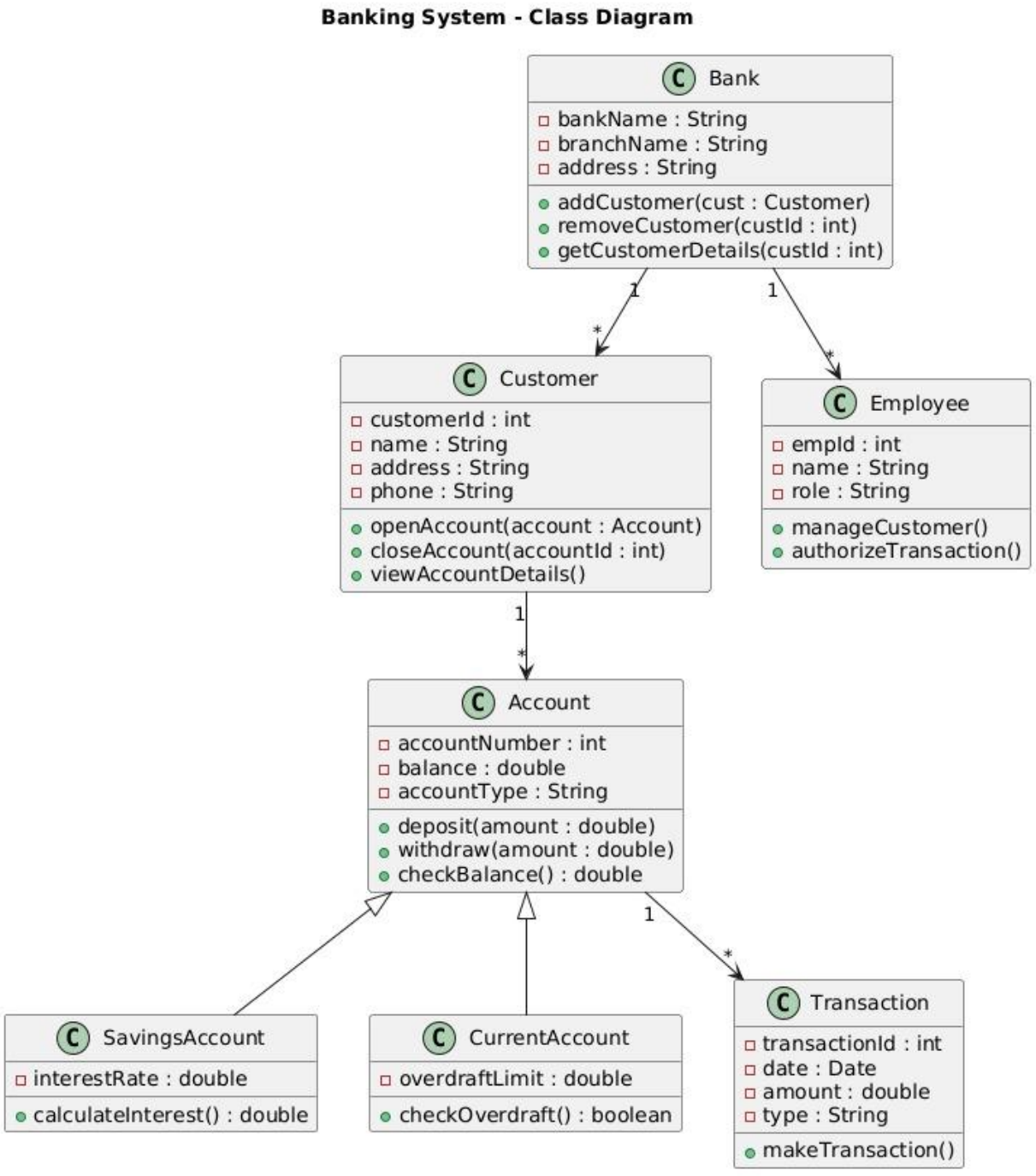
Object diagrams are dependent on the class diagram as they are derived from the class diagram. It represents an instance of a class diagram. The objects help in portraying a static view of an object-oriented system at a specific instant.

Both the object and class diagram are similar to some extent; the only difference is that the class diagram provides an abstract view of a system. It helps in visualizing a particular functionality of a system.

Notation of an Object Diagram



Class diagram For Banking System



PRACTICAL 5

Aim: To draw the behavioural view diagram: Sequence diagram.

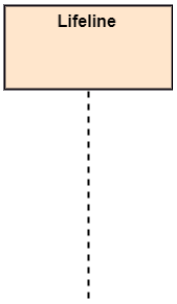
UML behavioural diagrams visualize, specify, construct, and document the dynamic aspects of a system. The behavioural diagrams are categorized as follows: use case diagrams, interaction diagrams, state–chart diagrams, and activity diagrams.

Sequence Diagrams :

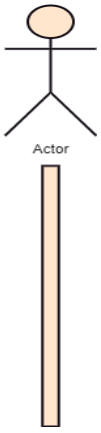
The sequence diagram represents the flow of messages in the system and is also termed as an event diagram. It helps in envisioning several dynamic scenarios. It portrays the communication between any two lifelines as a time-ordered sequence of events, such that these lifelines took part at the run time. In UML, the lifeline is represented by a vertical bar, whereas the message flow is represented by a vertical dotted line that extends across the bottom of the page. It incorporates the iterations as well as branching.

Notations of a Sequence Diagram :

1. **Lifeline :** An individual participant in the sequence diagram is represented by a lifeline. It is positioned at the top of the diagram.

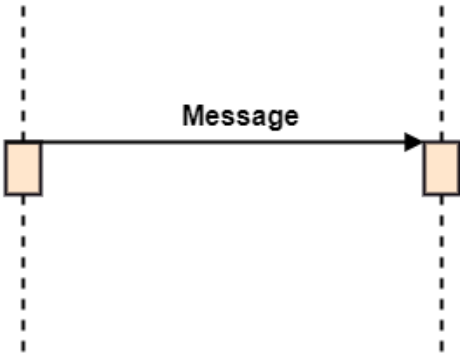


2. **Actor:** A role played by an entity that interacts with the subject is called as an actor. It is out of the scope of the system. It represents the role, which involves human users and external hardware or subjects. An actor may or may not represent a physical entity, but it purely depicts the role of an entity.

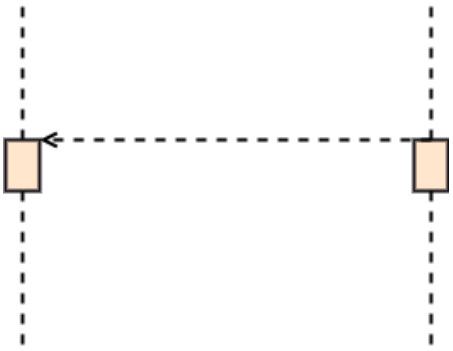


3. **Activation :** It is represented by a thin rectangle on the lifeline. It describes that time period in which an operation is performed by an element, such that the top and the bottom of the rectangle is associated with the initiation and the completion time, each respectively.
4. **Messages :** The messages depict the interaction between the objects and are represented by arrows. They are in the sequential order on the lifeline. The core of the sequence diagram is formed by messages and lifelines.
   Following are types of messages enlisted below:

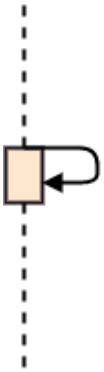
a. **Call Message** : It defines a particular communication between the lifelines of an interaction, which represents that the target lifeline has invoked an operation.



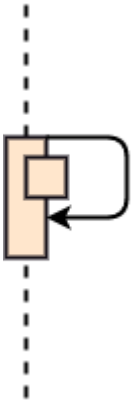
b. **Return Message :** It defines a particular communication between the lifelines of interaction that represent the flow of information from the receiver of the corresponding caller message.



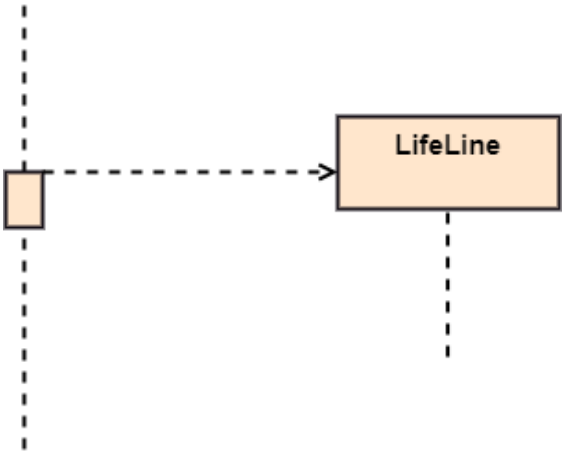
c. **Self Message:** It describes a communication, particularly between the lifelines of an interaction that represents a message of the same lifeline, has been invoked.



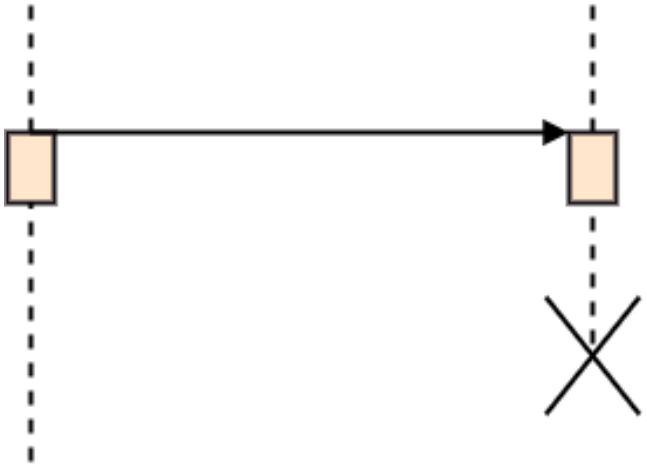
d. Recursive Message : A self message sent for recursive purpose is called a recursive message. In other words, it can be said that the recursive message is a special case of the self message as it represents the recursive calls.



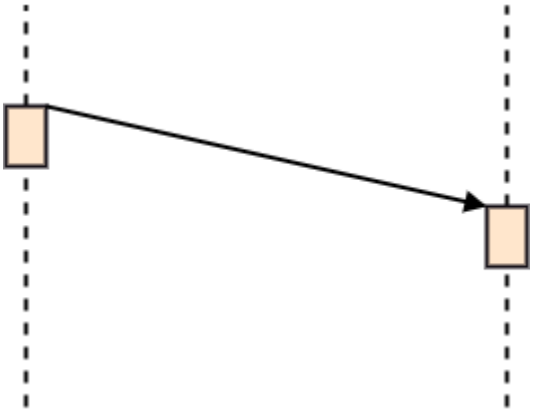
e. Create Message: It describes a communication, particularly between the lifelines of an interaction describing that the target (lifeline) has been instantiated.



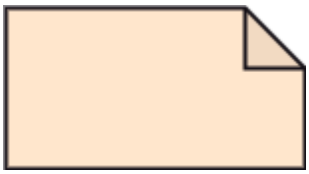
f. Destroy Message : It describes a communication, particularly between the lifelines of an interaction that depicts a request to destroy the lifecycle of the target.



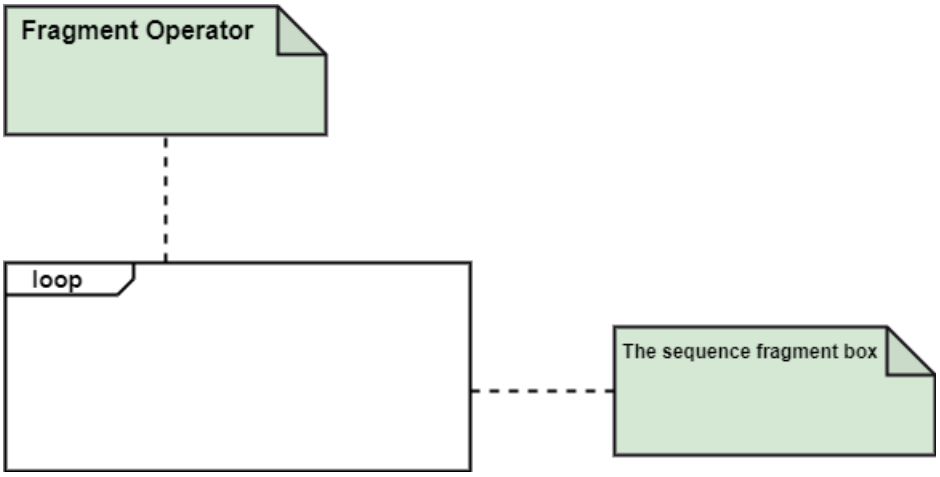
g. **Duration Message:** It describes a communication particularly between the lifelines of an interaction, which portrays the time passage of the message while modeling a system.



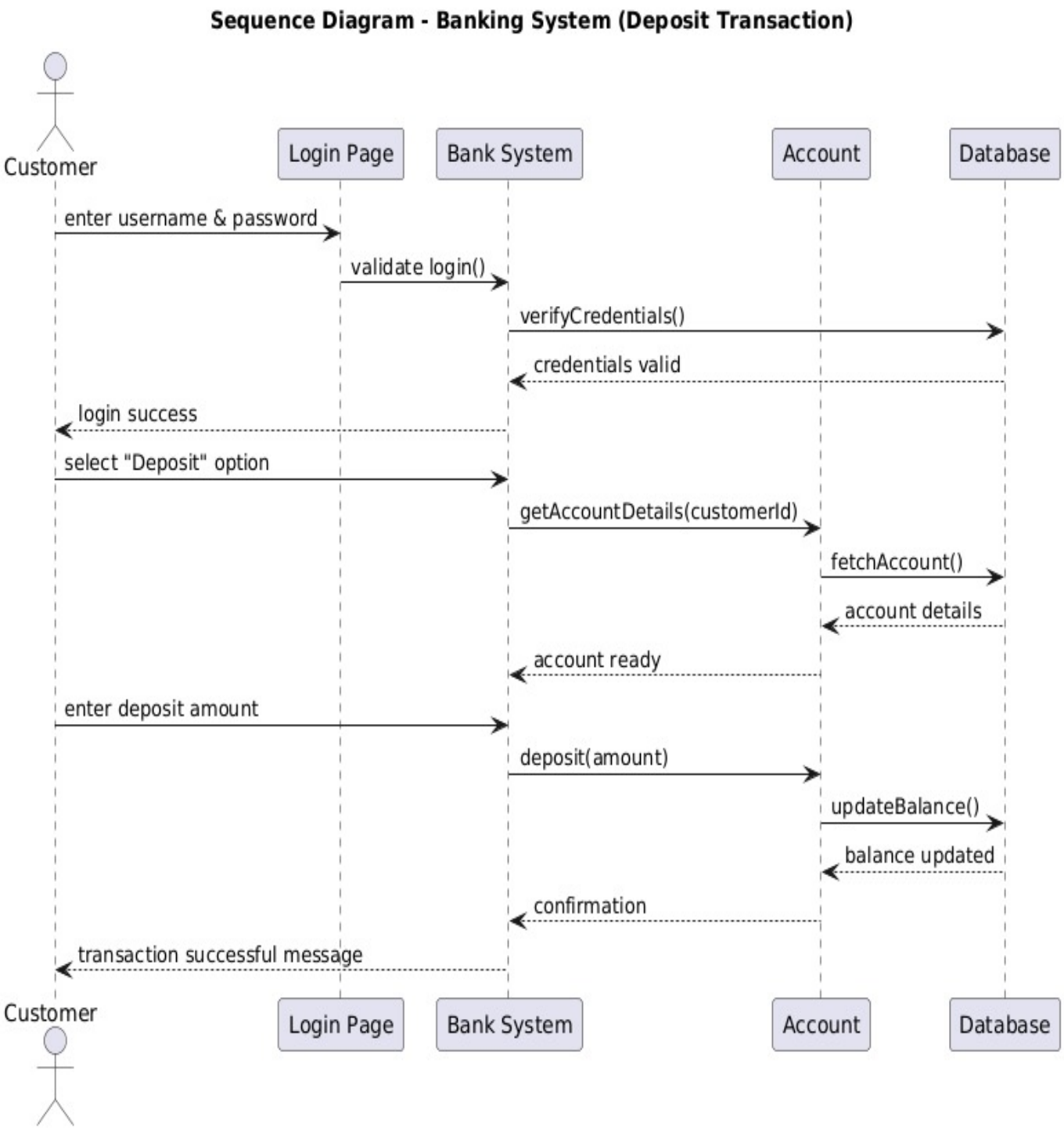
5. **Note :** A note is the capability of attaching several remarks to the element. It basically carries useful information for the modelers.



6. **Sequence Fragments :**Sequence fragments have been introduced by UML 2.0, which makes it quite easy for the creation and maintenance of an accurate sequence diagram. It is represented by a box called a combined fragment, encloses a part of interaction inside a sequence diagram.



Sequence diagram For Banking System



PRACTICAL 6

Aim: To draw the behavioral view diagram: State-chart diagram, Activity diagram.

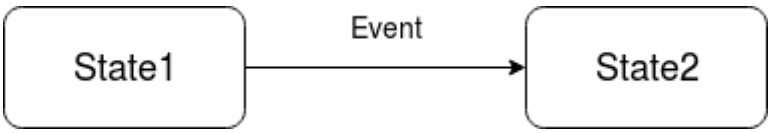
▪ State Diagrams

A state diagram is used to represent the condition of the system or part of the system at finite instances of time. It's a behavioral diagram and it represents the behavior using finite state transitions. State diagrams are also referred to as State machines and State-chart Diagrams. These terms are often used interchangeably.

Basic components of a state chart diagram –

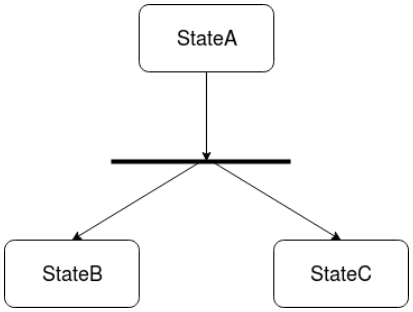
1. **Initial state** – We use a black filled circle represent the initial state of a System or a class.

2. **Transition** – We use a solid arrow to represent the transition or change of control from one state to another. The arrow is labelled with the event which causes the change in state.

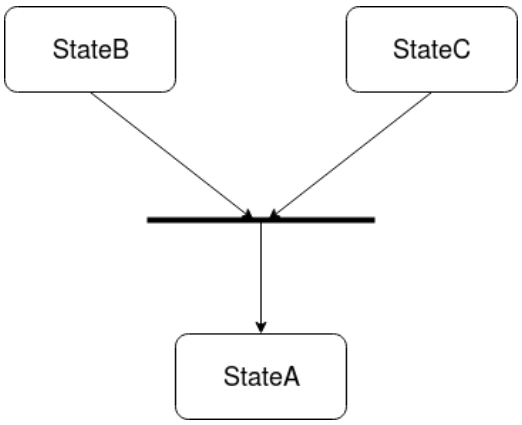


3. **State** – We use a rounded rectangle to represent a state. A state represents the conditions or circumstances of an object of a class at an instant of time.

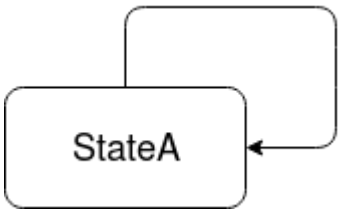
4. **Fork –** We use a rounded solid rectangular bar to represent a Fork notation with incoming arrow from the parent state and outgoing arrows towards the newly created states. We use the fork notation to represent a state splitting into two or more concurrent states.



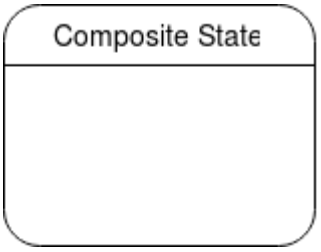
5. **Join** – We use a rounded solid rectangular bar to represent a Join notation with incoming arrows from the joining states and outgoing arrow towards the common goal state. We use the join notation when two or more states concurrently converge into one on the occurrence of an event or events.



6. **Self transition** – We use a solid arrow pointing back to the state itself to represent a self-transition. There might be scenarios when the state of the object does not change upon the occurrence of an event. We use self transitions to represent such cases.



7. **Composite state** – We use a rounded rectangle to represent a composite state also. We represent a state with internal activities using a composite state.



8. **Final state** – We use a filled circle within a circle notation to represent the final state in a state machine diagram.

Activity diagram:

The activity diagram is used to demonstrate the flow of control within the system rather than the implementation. It models the concurrent and sequential activities.

The activity diagram helps in envisioning the workflow from one activity to another. It put emphasis on the condition of flow and the order in which it occurs. The flow can be sequential, branched, or concurrent, and to deal with such kinds of flows, the activity diagram has come up with a fork, join, etc.

It is also termed as an object-oriented flowchart. It encompasses activities composed of a set of actions or operations that are applied to model the behavioral diagram.

Components of an Activity Diagram:

Following are the component of an activity diagram:

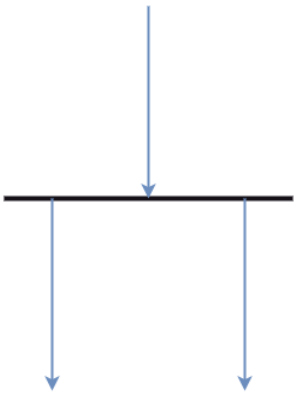
1. Activities:

The categorization of behavior into one or more actions is termed as an activity. In other words, it can be said that an activity is a network of nodes that are connected by edges. The edges depict the flow of execution. It may contain action nodes, control nodes, or object nodes.

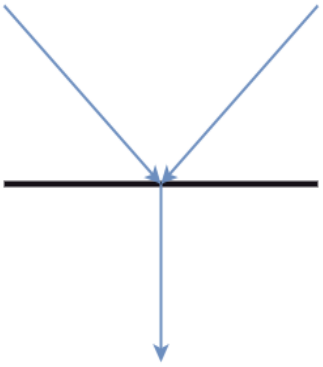
The control flow of activity is represented by control nodes and object nodes that illustrates the objects used within an activity. The activities are initiated at the initial node and are terminated at the final node.

2. Forks

Forks and join nodes generate the concurrent flow inside the activity. A fork node consists of one inward edge and several outward edges. It is the same as that of various decision parameters. Whenever a data is received at an inward edge, it gets copied and split crossways various outward edges. It split a single inward flow into multiple parallel flows.



3. **Join Nodes:** Join nodes are the opposite of fork nodes. A Logical AND operation is performed on all of the inward edges as it synchronizes the flow of input across one single output (outward) edge**.**

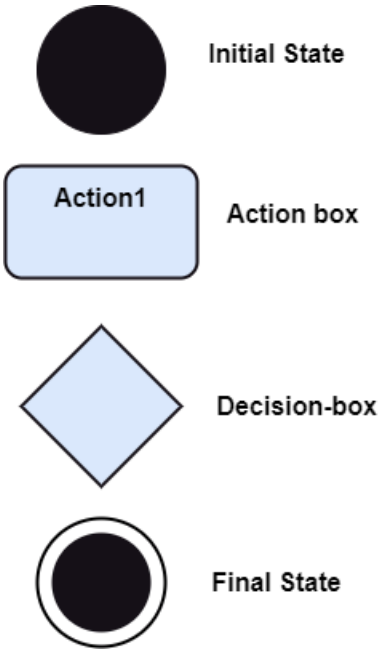


4. **Pins:** It is a small rectangle, which is attached to the action rectangle. It clears out all the messy and complicated thing to manage the execution flow of activities. It is an object node that precisely represents one input to or output from the action.

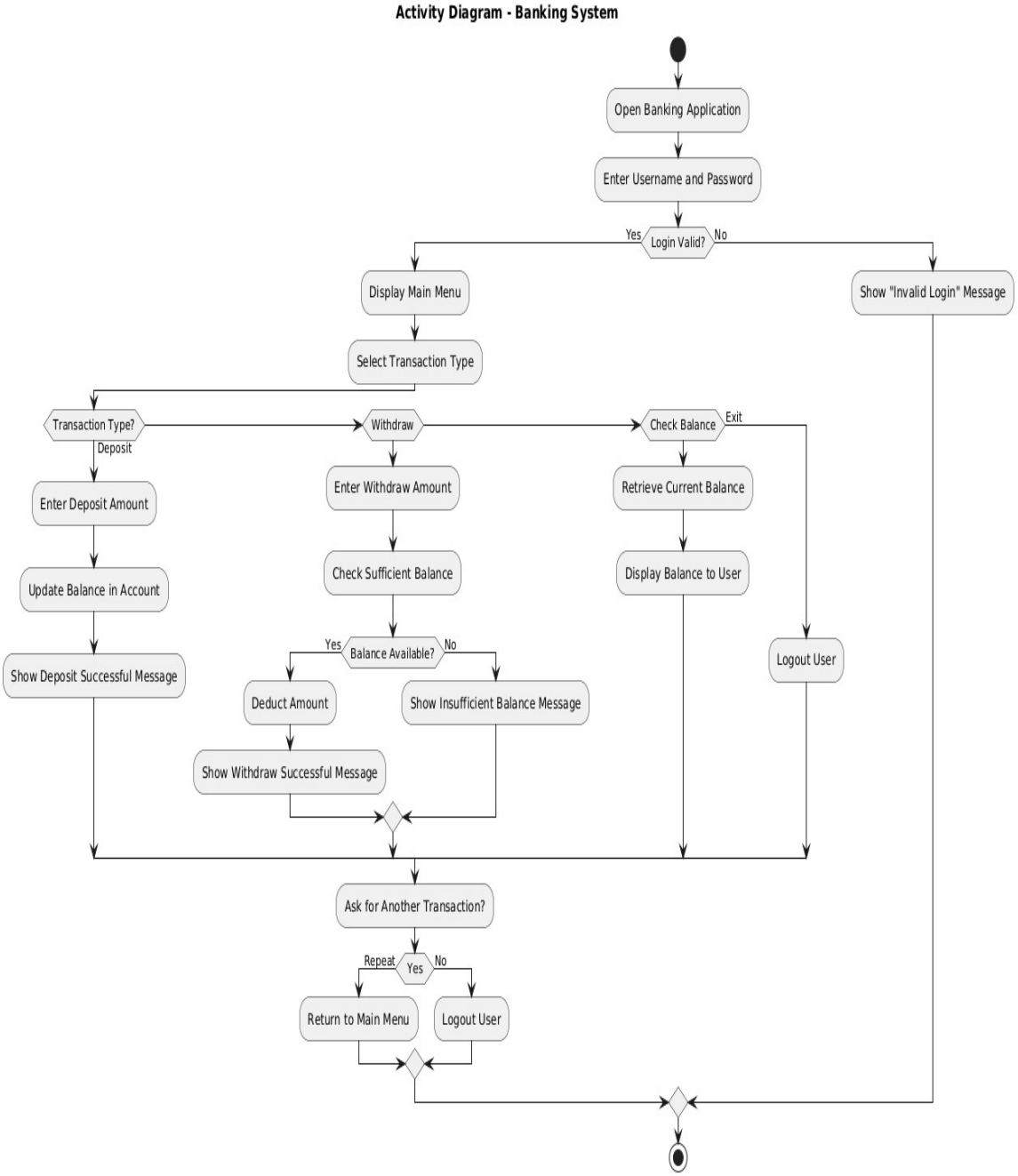
Notation of an Activity diagram:

Activity diagram constitutes following notations:

- **Initial State**: It depicts the initial stage or beginning of the set of actions.
- **Final State:** It is the stage where all the control flows and object flows end.
- **Decision Box**: It makes sure that the control flow or object flow will follow only one path.
- **Action Box**: It represents the set of actions that are to be performed.



Activity Diagram for Banking System



PRACTICAL 7

Aim: To perform the function oriented diagram: DFD and Structured chart

Function Oriented design is a method to software design where the model is decomposed into a set of interacting units or modules where each unit or module has a clearly defined function. Thus, the system is designed from a functional viewpoint.

Data Flow Diagrams :

A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It can be manual, automated, or a combination of both.

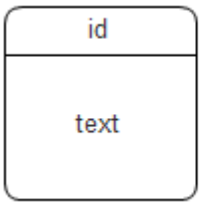
It shows how data enters and leaves the system, what changes the information, and where data is stored.

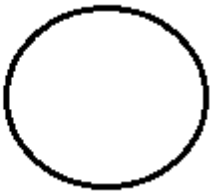
The objective of a DFD is to show the scope and boundaries of a system as a whole. It may be used as a communication tool between a system analyst and any person who plays a part in the order that acts as a starting point for redesigning a system. The DFD is also called as a data flow graph or bubble chart.

Data Flow Diagram Symbols :

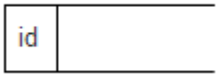
DFD can represent Source, destination, storage and flow of data using the following set of components –

Process: This represents any task that is performed by the system on the input data. It results in a processed output of data. A process can be related to the modification, storing, deletion, creation, or any other operation on data. It is represented by a circle or a round-edged rectangle.



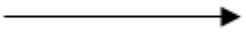


Database: This is where data is stored in a system. It can also be the input or output of a process as well. It is represented by a rectangle, which can sometimes have a small section for its ID.



Entities: They are either the source or the termination of the DFD. Ideally, they represent an external source from where data is taken or processed in the end. Entities are represented by a closed square/rectangle.

Data Flow: Lastly, we use directional arrows to represent the flow of data from one entity/database to another. The line would have a directional arrow in the end to depict the source and the destination units.

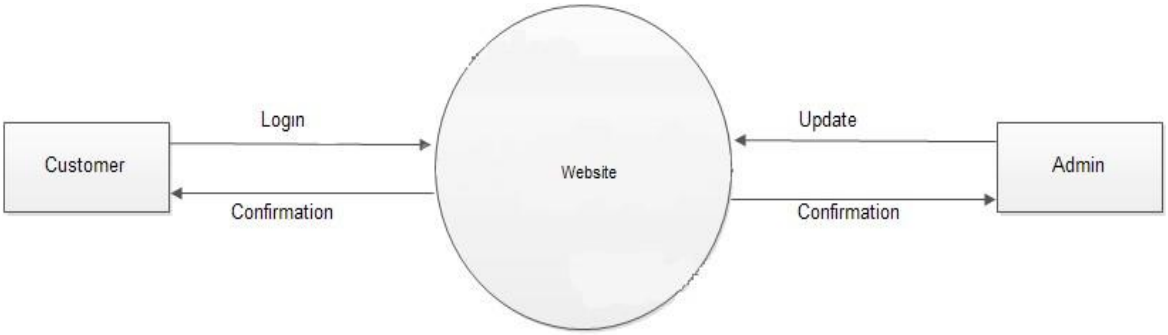


Data Flow Diagram Levels:

Data Flow Diagrams are composed of levels and layers, from the basic to the higher level. The level of the diagram depends upon the scope of the process. As the level increases, the complexity of the diagram increases. The high-level Data Flow Diagrams are categorized into low levels with more detailed process information.

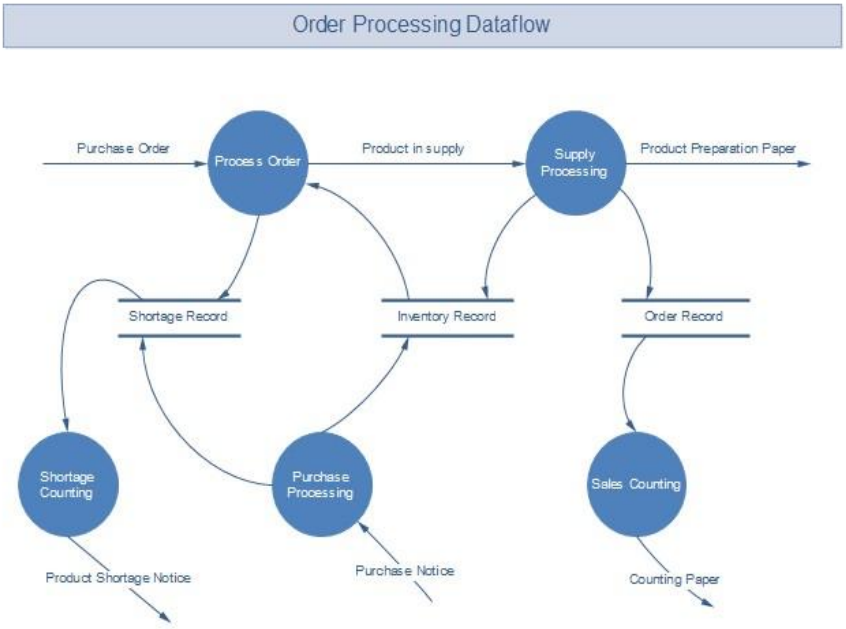
Level 0 DFDs :

The Level 0 data flow diagram gives a general overview of the process. They are also called context diagrams. The level 0 DFDs depict the single process node and its links with external entities. It is understood by all the stakeholders participating in the activity.



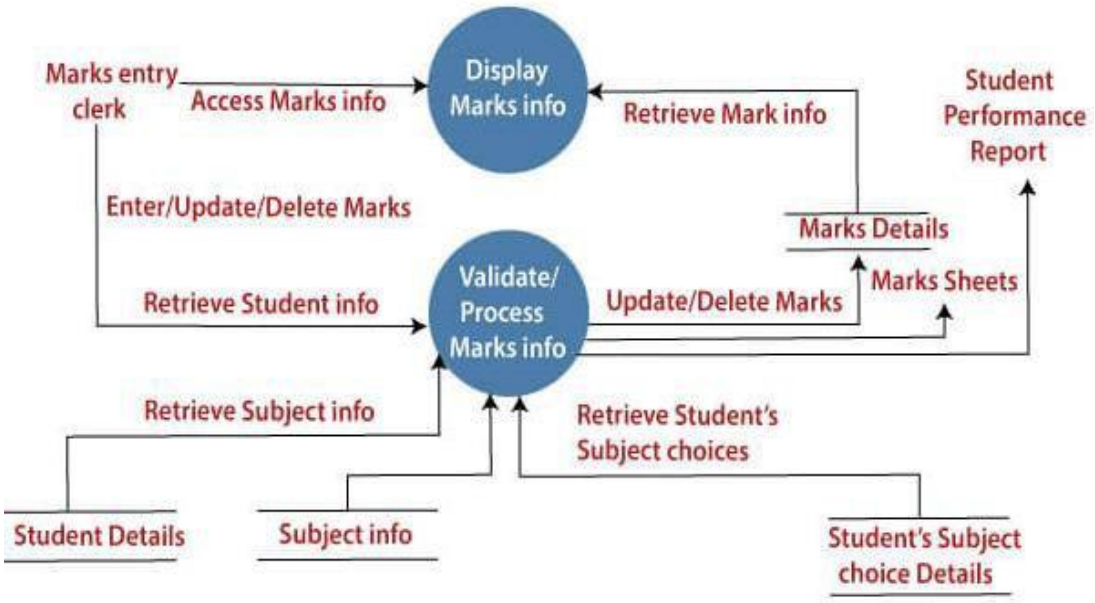
Level 1 DFDs :

The Level 1 data flow diagrams are the more detailed breakout of parts of the Level 0 DFDs. In a level 1 DFD, the single process node is broken down into sub-processes with additional data flows and data stores to connect them.

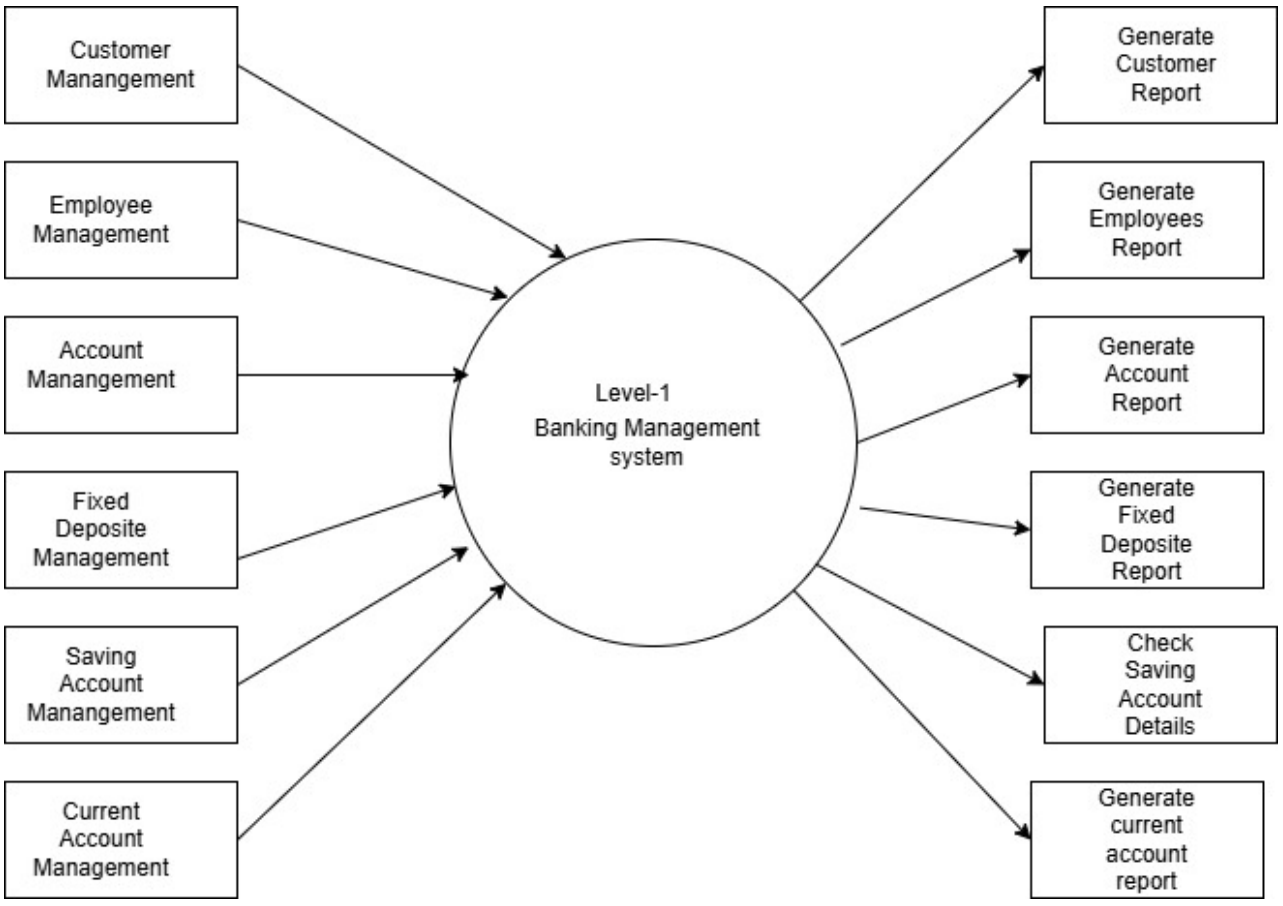


Level 2 DFDs :

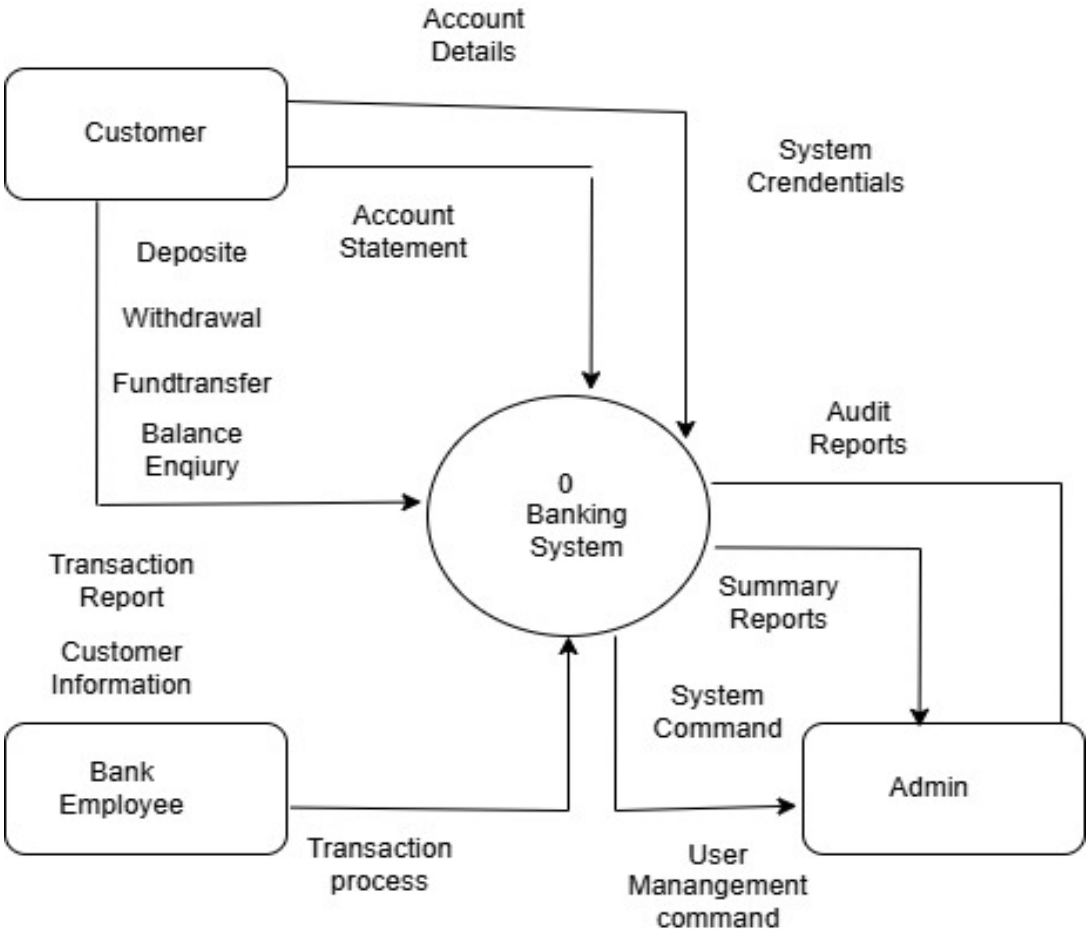
The Level 2 data Flow diagrams go into deeper sub-processes to better illustrate a process and its stages. More symbols, shapes, and text are required to detail the process depicting the system's functioning.



DFD for Bank System: Level 0: Context Diagram



Level 1: DFD Banking System



PRACTICAL 8

Aim: To draw the environmental view diagram: Deployment diagram.

The deployment diagram visualizes the physical hardware on which the software will be deployed. It portrays the static deployment view of a system. It involves the nodes and their relationships.

It ascertains how software is deployed on the hardware. It maps the software architecture created in design to the physical system architecture, where the software will be executed as a node. Since it involves many nodes, the relationship is shown by utilizing communication paths.

Purpose of Deployment Diagram :

The main purpose of the deployment diagram is to represent how software is installed on the hardware component. It depicts in what manner a software interacts with hardware to perform its execution.

Both the deployment diagram and the component diagram are closely interrelated to each other as they focus on software and hardware components. The component diagram represents the components of a system, whereas the deployment diagram describes how they are actually deployed on the hardware.

The deployment diagram does not focus on the logical components of the system, but it put its attention on the hardware topology.

Following are the purposes of deployment diagram enlisted below:

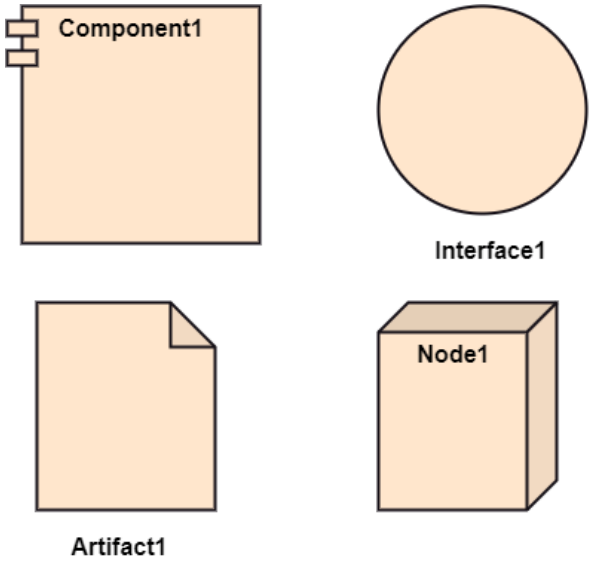
1. To envision the hardware topology of the system.
2. To represent the hardware components on which the software components are installed.
3. To describe the processing of nodes at the runtime.

Symbol and notation of Deployment diagram

The deployment diagram consist of the following notations:

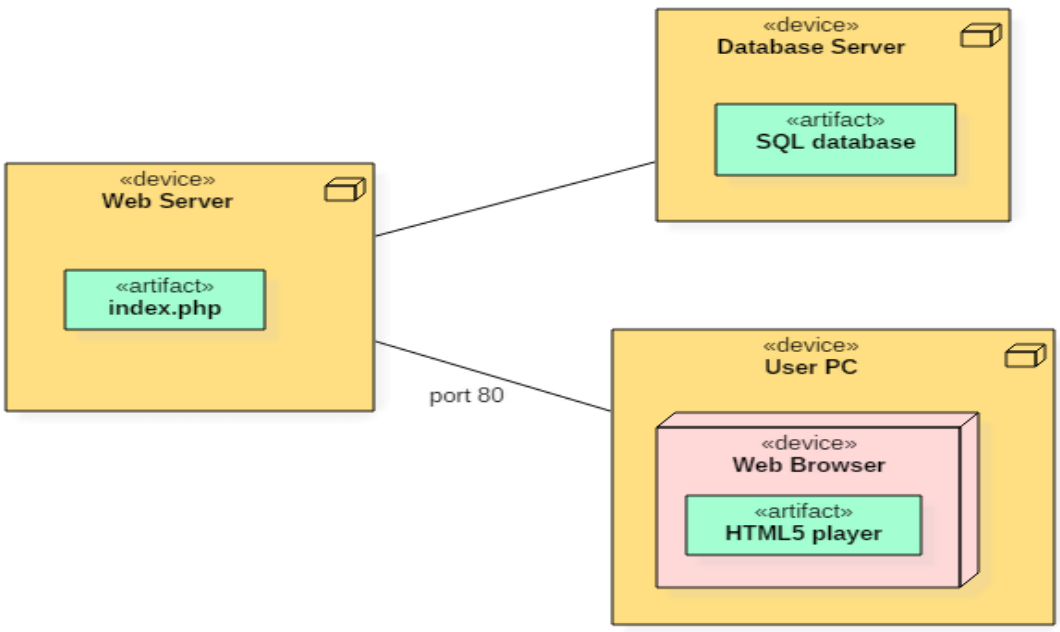
- A component

- An artifact
- An interface
- A node

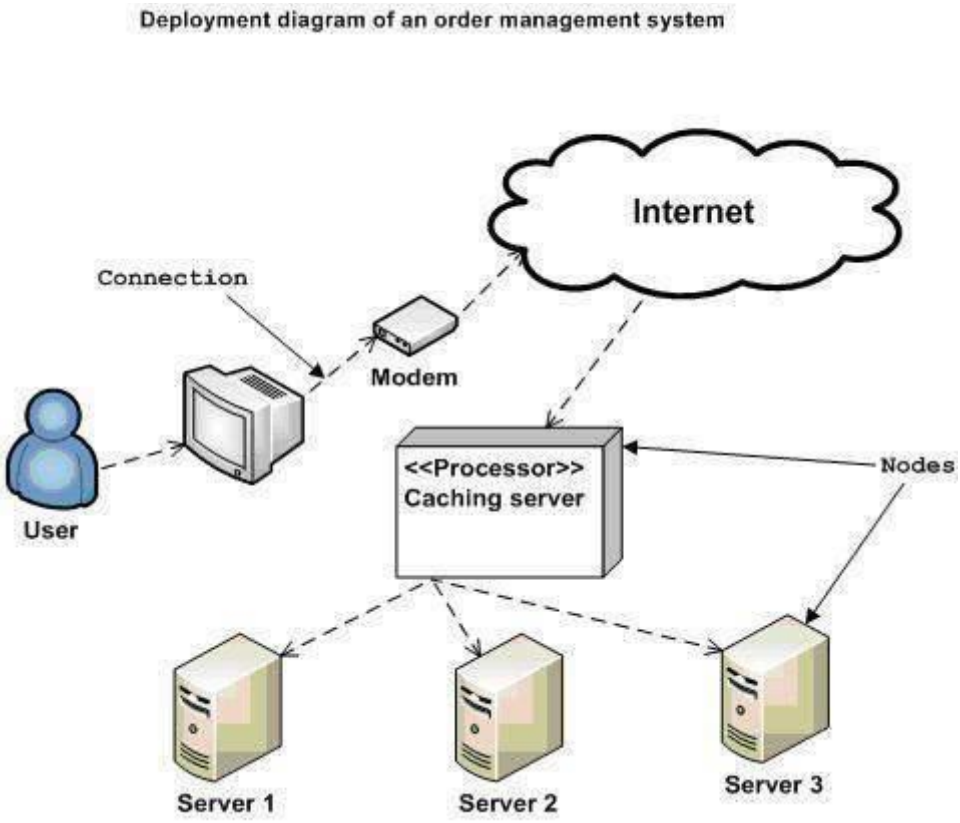


Example of a Deployment diagram

Example 1 : Following deployment diagram represents the working of HTML5 video player in the browser:



Example 2 : The following deployment diagram of an order management system.



Deployment diagrams can be used for the followings:

- To model the network and hardware topology of a system.
- To model the distributed networks and systems.
- Implement forwarding and reverse engineering processes.
- To model the hardware details for a client/server system.
- For modeling the embedded system

PRACTICAL 9

Aim: Study of any two Open source tools in DevOps.

DevOps is a set of practices, tools, and a cultural philosophy that automate and integrate the processes between software development and IT teams. It emphasizes team empowerment, cross-team communication and collaboration, and technology automation.

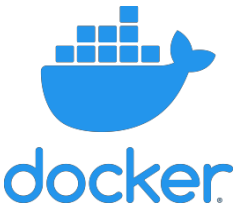
The DevOps movement began around 2007 when the software development and IT operations communities raised concerns about the traditional software development model, where developers who wrote code worked apart from operations who deployed and supported the code. The term DevOps, a combination of the words development and operations, reflects the process of integrating these disciplines into one, continuous process.

DevOps isn't just a cultural shift — it requires great tools to come to fruition. Below, we've pulled together a list of some of the most well-loved DevOps tools available today. But, throwing loads of money into fancy SaaS solutions can quickly gobble up the cloud budget. These DevOps tools all are open source, and enable everything from container builds and orchestration to microservices networking, configuration management, CI/CD automation, full-stack monitoring and more.

Most popular DevOps tools for Infrastructure automation and Monitoring are given below.

1. Jenkins
2. Puppet
3. Chef
4. Vagrant
5. Docker
6. Saltstack
7. Ansible
8. Juju
9. Sensu
10. Consul
11. New Relic
12. Monit

	Jenkins is a Java-based continuous integration tool used for orchestration for the application provisioning and deployment. It has to be associated with a version control system (SVN) when you want a new code to push into the code repository. The Jenkins server allows you to build and test the new code and notifies the team about the process, changes and final results. Jenkins as an orchestration tool used for building pipelines. You can also configure polls for any comments on SVN and trigger builds and regression tests on the new code that is added according to the latest code commits.
	Puppet is a ruby based configuration management tool. It is written with puppet DSL and is wrapped in modules. Puppet is developed with special consideration of system administrators. The design of Puppet felicitates fast deployment, efficiently manages the infrastructure as code with a task-based approach. Which improves the overall quality of the software. It runs a puppet agent running on all servers that need to be configured. Instead of running any codes on the infrastructure it builds a graph that represents the code base of the infrastructure. It will also figure out the best possibilities to achieve the desired state. Puppet pulls a complied module with the specification desired and installs the software needed to complete the tasks. You can find all the puppet community modules called Puppet forge here.
	Chef is one of the most popular agent- based cloud infrastructure automation tool for configuration management. Ruby-based DSL is used, you have to code your infrastructure and define the configuration scenarios.. Chef is helpful to maintain consistent configuration, this reduces the operating costs, with effective data centers management. And it also offers customization, flexibility, and readable configuration policies. The tools provide the support of simple migration, allows you to integrate with multiple platforms such as FreeBSD and AIX. Chef provides you with a rich selection of ready to use templates. You can find everything you need on the community cookbook here.

	Vagrant is another most popular tool for configuring virtual machines for your development environment. The tools run on the top of VM solutions such as Virtualbox HyperV etc. Vagrant can handle all the configurations with help for Vagrantfile. This Vagrantfile contains all the configurations required for the Virtual machines. You can create a configuration, it can be shared with other developers to imitate the process of the same configuration. Vagrant also provides plugins for cloud provisioning, infrastructure automation tools.
	Docker will be another most preferred tools for continuous integration and deployment. Docker is an automation tool build on the top of Linux containers with the support for containerization. Docker works on the concept of process-level virtualization and creates isolated environments for applications which are known as containers. The containers can be easily shipped to another server without any changes made to the application, which further doesn't have any OS-related dependencies. Docker is predominantly used by DevOps professionals in cloud computing and has active community support on both Windows and Linux.
	Saltsack is a Python-based configuration management tool. Unlike like few other tools which pull the code for configuration from the server, Saltstack pushes the code to various node simultaneously. It compiles the code faster than Puppet and Chef and the configuration of the code is also pretty quick.

PRACTICAL 10

Aim: Estimating effort using FP Estimation for chosen system

Effort estimation :The managers estimate efforts in terms of personnel requirement and man-hour required to produce the software. For effort estimation software size should be known. This can either be derived by managers' experience, organization's historical data or software size can be converted into efforts by using some standar formulae.

A Function Point (FP) is a unit of measurement to express the amount of business functionality, an information system (as a product) provides to a user. FPs measure software size. They are widely accepted as an industry standard for functional sizing.

It has Five Main Components: Data Functions

Internal Logical Files External Interface Files

Transactional Functions

External Inputs External Outputs External Inquiries

Internal Logical Files –

The first data function allows users to utilize data they are responsible for maintaining. For example, a pilot may enter navigational data through a display in the cockpit prior to departure. The data is stored in a file for use and can be modified during the mission.

Therefore the pilot is responsible for maintaining the file that contains the navigational information. Logical groupings of data in a system, maintained by an end user, are referred to as Internal Logical Files (ILF).

External Interface Files –

The second Data Function a system provides an end user is also related to logical groupings of data. In this case the user is not responsible for maintaining the data. The data resides in another system and is maintained by another user or system. The user of the system being counted requires this data for reference purposes only. For example, it may be necessary for a pilot to reference position data from a satellite or

ground-based facility during flight. The pilot does not have the responsibility for updating data at these sites but must reference it during the flight. Groupings of data from another system that are used only for reference purposes are defined as External Interface Files (EIF).

The remaining functions address the user's capability to access the data contained in ILFs and EIFs. This capability includes maintaining, inquiring and outputting of data. These are referred to as Transactional Functions.

External Input –

The first Transactional Function allows a user to maintain Internal Logical Files (ILFs) through the ability to add, change and delete the data. For example, a pilot can add, change and delete navigational information prior to and during the mission. In this case the pilot is utilizing a transaction referred to as an External Input (EI). An External Input gives the user the capability to maintain the data in ILF's through adding, changing and deleting its contents.

External Output –

The next Transactional Function gives the user the ability to produce outputs. For example a pilot has the ability to separately display ground speed, true air speed and calibrated air speed. The results displayed are derived using data that is maintained and data that is referenced. In function point terminology the resulting display is called an External Output (EO).

External Inquiries –

The final capability provided to users through a computerized system addresses the requirement to select and display specific data from files. To accomplish this user inputs selection information that is used to retrieve data that meets the specific criteria. In this situation there is no manipulation of the data. It is a direct retrieval of information contained on the files. For example if a pilot displays terrain clearance data that was previously set, the resulting output is the direct retrieval of stored information. These transactions are referred to as External Inquiries (EQ).

Function Point Analysis :

The basic and primary purpose of the functional point analysis is to measure and provide the software application functional size to the client, customer, and the

stakeholder on their request. Further, it is used to measure the software project development along with its maintenance, consistently throughout the project irrespective of the tools and the technologies.

For Computing Function Points

$$\mathbf{FP = Count\ Total * [\ 0.65 + 0.01 * \sum(F_i)\ ]}$$

Count Total is the sum of all FP entries

Fi (i=1 to 14) are complexity value adjustment factors (VAF).

Information Domain Value	Count		Weighting factor Simple	Average	Complex		
External Inputs (EIs)		×	3	4	6	=	
External Outputs (EOs)		×	4	5	7	=	
External Inquiries (EQs)		×	3	4	6	=	
Internal Logical Files (ILFs)		×	7	10	15	=	
External Interface Files (EIFs)		×	5	7	10	=	
Count total	→						

PRACTICAL 11

Aim: Study of any one Testing tool

Software Testing Tools:

Software Testing tools are the tools that are used for the testing of software. Software testing tools are often used to assure firmness, thoroughness, and performance in testing software products. Unit testing and subsequent integration testing can be performed by software testing tools. These tools are used to fulfill all the requirements of planned testing activities. These tools also work as commercial software testing tools. The quality of the software is evaluated by software testers with the help of various testing tools**.**

→ Types of Testing Tools

Software testing is of two types, static testing, and dynamic testing. Also, the tools used during these testing are named accordingly on these testings. Testing tools can be categorized into two types which are as follows:

1. Static Test Tools: Static test tools are used to work on the static testing processes. In the testing through these tools, the typical approach is taken. These tools do not test the real execution of the software. Certain input and output are not required in these tools. Static test tools consist of the following:

a) **Flow analyzers:** Flow analyzers provides flexibility in the data flow from input to output.
b) **Path Tests:** It finds the not used code and code with inconsistency in the software.
c) Coverage Analyzers: All rationale paths in the software are assured by the coverage analyzers.
d) **Interface Analyzers:** They check out the consequences of passing variables and data in the modules.

2. Dynamic Test Tools: Dynamic testing process is performed by the dynamic test tools. These tools test the software with existing or current data. Dynamic test tools comprise the following:

a) **Test driver:** The test driver provides the input data to a module-under-test (MUT).
b) **Test Beds**: It displays source code along with the program under execution at the same time.
c) **Emulators**: Emulators provide the response facilities which are used to imitate parts of the system not yet developed.
d) **Mutation Analyzers**: They are used for testing the fault tolerance of the system by knowingly providing the errors in the code of the software.

There is one more categorization of software testing tools. According to this classification, software testing tools are of 10 types:

1) **Test Management Tools**: Test management tools are used to store information on how testing is to be done, help to plan test activities, and report the status of quality assurance activities. For example, JIRA, Redmine, Selenium, etc.
2) **Automated Testing Tools**: Automated testing tools helps to conduct testing activities without human intervention with more accuracy and less time and effort. For example, Appium, Cucumber, Ranorex, etc.
3) **Performance Testing Tools**: Performance testing tools helps to perform effectively and efficiently performance testing which is a type of non-functional testing that checks the application for parameters like stability, scalability, performance, speed, etc. For example, WebLOAD, Apache JMeter, Neo Load, etc.
4) **Cross-browser Testing Tools:** Cross-browser testing tools helps to perform cross-browser testing that lets the tester check whether the website works as intended when accessed through different browser-OS combinations. For example, Testsigma, Testim, Perfecto, etc.
5) **Integration Testing Tools:** Integration testing tools are used to test the interface between the modules and detect the bugs. The main purpose here is to check whether the specific modules are working as per the client's needs or not. For example, Citrus, FitNesse, TESSY, etc.

6) **Unit Testing Tools**: Unit testing tools are used to check the functionality of individual modules and to make sure that all independent modules works as expected. For example, Jenkins, PHPUnit, JUnit, etc.

7) **Mobile Testing Tools:** Mobile testing tools are used to test the application for compatibility on different mobile devices. For example, Appium, Robotium, Test IO, etc.
8) **GUI Testing Tools:** GUI testing tools are used to test the graphical user interface of the software. For example, EggPlant, Squish, AutoIT, etc.
9) **Bug Tracking Tools**: Bug tracking tool helps to keep track of various bugs that come up during the application lifecycle management. It helps to monitor and log all the bugs that are detected during software testing. For example, Trello, JIRA, GitHub, etc.
10) **Security Testing Tools**: Security testing is used to detect the vulnerabilities and safeguard the application against the malicious attacks. For example, NetSparker, Vega, ImmuniWeb, etc.

→ Automation Testing.

The techniques used to perform different kinds of testing and test suits to check the compatibility of the software can be termed automation testing. This testing requires minimal or no human intervention and in case of any bugs or errors, that issue can be addressed before the issue arises. However, automation testing and their testing tools can be written in JavaScript, Ruby, or C#.

→ Popular Automation Tools

- **Selenium:** Selenium is an automated testing tool that is used for Regression testing and provides a playback and recording facility. It can be used with frameworks like JUnit and Test-NG . It provides a single interface and lets users write test cases in languages like Ruby, Java, Python, etc.
- **QTP:** Quick Test Professional (QTP) is an automated functional testing tool to test both web and desktop applications. It is based on the VB scripting language and it provides functional and regression test automation for software applications.

- **Sikuli:** It is a GUI-based test automation tool that is used for interacting with elements of web pages. It is used to search and automate graphical user interfaces using screenshots.

- **Appium:** Apium is an open-source test automation framework that allows QAs to conduct automated app testing on different platforms like iOS, Android, and Windows SDK.
- **Jmeter:** Apache JMeter is an open-source Java application that is used to load test the functional behavior of the application and measure the performance.

→ *Selenium:*

Selenium is a widely used tool for testing web-based applications that checks if they are doing as expected. It is a prominent preference amongst testers for cross-browser testing and is viewed as one of the most reliable systems for web application automation evaluation. Selenium is also platform-independent, so it can provide distributed testing using the Selenium Network. Selenium is a powerful tool for controlling web browsers through programs and performing browser automation. It is functional for all browsers, works on all major OS and its scripts are written in various languages.

Fig: Features of selenium

→ Applications of selenium

Selenium is an open-source tool for automating web browsers. It helps to test the website functionality easily and check the regular performance across different browsers and systems through cross-browser testing.

- **Automated Web Application Testing**: Selenium is mainly used for the automation testing of web applications across different browsers and platforms.
- **Cross-Browser Testing**: It checks the compatibility of web applications across various web browsers like Chrome, Firefox, Safari, and Edge.
- **Web Scraping**: Automates the filter of the data from websites for purposes such as data analysis and monitoring with easy automation.
- **Continuous Integration/Continuous Deployment (CI/CD)**: Integrates with CI/CD tools like Jenkins to enable continuous testing as part of the development pipeline and automation.
- **Functional Testing:** It checks the functionality of a web application against the specified requirements of the browser.